

Machine Learning

From first principles to working models, without the
scary maths

Sample study notes · Knowledge Dungeon

Contents

What Is Machine Learning?	3
Working with Data	13
Linear Regression	22
Classification and Logistic Regression	31
Evaluating Models	39
Decision Trees and Ensembles	48
Nearest Neighbours and Support Vector Machines	57
Unsupervised Learning: Clustering and Dimensionality Reduction	66
Neural Networks and Deep Learning	75
Machine Learning in Practice and Ethics	84

What Is Machine Learning?

Your email app moves spam out of your inbox. Your music app suggests songs you end up loving. Your phone unlocks when it sees your face. None of these programs were given a list of rules by a programmer. Instead, they **learned** from examples. This chapter explains what that means and introduces the main ideas you will use throughout the book.

Traditional Programming vs Machine Learning

In **traditional programming**, a person writes rules, the computer applies them to data, and out come answers.

```
Rules + Data -> Program -> Answers
```

To detect spam the traditional way, you might write: "if the email contains 'lottery' and 'winner', mark it as spam". This breaks quickly. Spammers change their words, and you end up with thousands of fragile rules.

In **machine learning**, we give the computer data *and* the correct answers, and it works out the rules itself.

```
Data + Answers -> Learning algorithm -> Model (the rules)
```

The result is a **model**: a function that takes new inputs and produces predictions. Show it thousands of emails labelled

"spam" or "not spam", and it learns which patterns matter.

Key idea: Machine learning is the study of algorithms that improve their performance at a task through experience (data), rather than through explicitly programmed rules.

A classic definition by Tom Mitchell puts it precisely: a program learns from **experience E** with respect to a **task T** and **performance measure P** if its performance at T, measured by P, improves with E. For spam filtering, T is classifying emails, E is a set of labelled emails, and P is the percentage classified correctly.

When Should You Use Machine Learning?

Machine learning shines when:

- the rules are too complex to write by hand (recognising faces, understanding speech),
- the rules keep changing (spam, fraud, trending products),
- the problem involves huge amounts of data that no person could study, or
- we want to discover patterns we did not know existed (grouping customers).

It is a poor choice when a simple rule works perfectly, when there is little or no data, or when every decision must be fully explainable and a formula already exists. You do not need

machine learning to calculate GST.

Types of Machine Learning

Supervised Learning

In **supervised learning**, every training example comes with the correct answer, called the **label**. The model learns to map inputs to labels.

- **Classification:** the label is a category. Is this email spam or not? Is this tumour benign or malignant? Which digit is in this image?
- **Regression:** the label is a number. What will this house sell for? How many units will we sell next month?

Unsupervised Learning

In **unsupervised learning**, there are no labels. The model looks for structure on its own.

- **Clustering:** grouping similar items, such as customers with similar shopping habits.
- **Dimensionality reduction:** compressing many features into a few while keeping most of the information.
- **Anomaly detection:** spotting unusual items, such as a fraudulent transaction.

Reinforcement Learning

In **reinforcement learning**, an **agent** takes actions in an **environment** and receives **rewards** or penalties. It learns a strategy (a **policy**) that maximises total reward over time. Game-playing programs, robot control and some recommendation systems use reinforcement learning.

Semi-Supervised and Self-Supervised Learning

Labels are often expensive, because someone must hand-label every example. **Semi-supervised learning** combines a small labelled set with a large unlabelled one. **Self-supervised learning** creates labels from the data itself, for example by hiding a word in a sentence and asking the model to predict it. Large language models are trained this way.

Type	Has labels?	Example task
Supervised	Yes	Predict house prices
Unsupervised	No	Group customers
Reinforcement	Rewards, not labels	Learn to play chess
Self-supervised	Created from data	Predict the next word

Key Vocabulary

- **Dataset:** the collection of examples used for learning.
- **Example / sample / instance:** one row of the dataset, such as one house.

- **Feature:** an input variable describing an example, such as area, number of bedrooms or location. Features are usually written x .
- **Label / target:** the value we want to predict, such as the price. Usually written y .
- **Model:** the learned function that maps features to predictions.
- **Parameters:** the numbers inside the model that are learned from data, such as weights.
- **Hyperparameters:** settings chosen *before* training, such as how many trees a forest has. They are not learned directly.
- **Training:** the process of adjusting parameters to fit the data.
- **Inference / prediction:** using a trained model on new data.

The Machine Learning Workflow

Real projects follow a fairly standard sequence of steps.

1. **Define the problem.** What exactly are we predicting, and how will we measure success? "Reduce customer churn" becomes "predict which customers will cancel within 30 days".

2. **Collect data.** Gather examples from databases, logs, surveys or public datasets.
3. **Explore and clean the data.** Look for missing values, errors and odd distributions (Chapter 2).
4. **Engineer features.** Choose and transform inputs that help the model.
5. **Split the data** into training, validation and test sets.
6. **Choose and train models.** Start simple, then try more powerful models.
7. **Evaluate.** Measure performance on data the model has never seen (Chapter 5).
8. **Tune.** Adjust hyperparameters to improve validation performance.
9. **Deploy.** Put the model into a real application.
10. **Monitor.** Watch performance over time, because the world changes and models go stale.

Key idea: In practice, most time goes into steps 2 to 4, the data work. A simple model on clean, relevant data usually beats a fancy model on messy data.

Generalization: The Real Goal

A model is only useful if it works on **new** data. Memorising the training examples is easy; performing well on unseen examples is the real challenge. This ability is called **generalization**.

- **Underfitting:** the model is too simple to capture the pattern. It performs badly even on the training data. Fitting a straight line to a curved relationship underfits.
- **Overfitting:** the model is so complex that it memorises noise in the training data. It looks excellent in training but fails on new data, like a student who memorises answers without understanding.
- **Good fit:** the model captures the real pattern and ignores the noise.

This is why we always keep a **test set** aside and never train on it. Its score estimates how the model will perform in the real world.

A Tiny Example in Python

The scikit-learn library makes simple models very short to write.

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier

X, y = load_iris(return_X_y=True)          # 150 flowers,
      4 features each
X_train, X_test, y_train, y_test = train_test_split(X, y,
      test_size=0.2, random_state=0)

model = DecisionTreeClassifier(max_depth=3)
model.fit(X_train, y_train)                # training
print(model.score(X_test, y_test))        # accuracy on
      unseen flowers
```

The whole ML workflow is visible here in miniature: load data, split it, train a model, and evaluate it on data it has not seen.

A Brief History

- **1950s:** Alan Turing asks whether machines can think; Arthur Samuel's checkers program learns from self-play and coins the term "machine learning".
- **1958:** Frank Rosenblatt builds the perceptron, an early neural network.
- **1980s-1990s:** decision trees, backpropagation for neural networks and support vector machines become popular.
- **2000s:** large datasets and ensemble methods such as random forests spread through industry.
- **2012 onwards:** deep learning, powered by GPUs and big data, transforms image recognition, speech and language.

- **2020s:** large language models and generative AI reach millions of everyday users.

Summary

- Machine learning learns rules from data and answers instead of having rules written by hand.
- Supervised learning uses labels (classification and regression); unsupervised learning finds structure without labels; reinforcement learning learns from rewards.
- Features are inputs, labels are targets, parameters are learned and hyperparameters are chosen.
- The workflow runs from problem definition through data work, training and evaluation to deployment and monitoring.
- The goal is generalization: avoid both underfitting and overfitting, and judge models on unseen test data.

Practice Questions

1. Explain the difference between traditional programming and machine learning with an example of your own.
2. Classify each as classification, regression or clustering: predicting tomorrow's temperature, detecting fraudulent payments, grouping news articles by topic.
3. What is the difference between a parameter and a hyperparameter?

4. Describe overfitting using a student-exam analogy.
5. Why must the test set never be used during training?

Working with Data

Data scientists often joke that they spend 80 percent of their time cleaning data and 20 percent complaining about it. The joke is close to the truth. Models learn only what the data shows them, so messy, biased or badly prepared data produces bad models no matter how clever the algorithm. This chapter covers the essential steps between raw data and a dataset ready for training.

Kinds of Data

Features come in a few basic types, and each needs different handling.

- **Numerical (quantitative):** numbers that can be measured.
 - *Continuous:* any value in a range, such as height or temperature.
 - *Discrete:* countable values, such as number of rooms.
- **Categorical (qualitative):** labels or groups.
 - *Nominal:* no natural order, such as city or colour.
 - *Ordinal:* a meaningful order, such as small / medium / large, or grade A / B / C.

- **Text, images, audio and time series:** unstructured data that must be converted into numbers before most models can use it.

Exploring the Data

Before changing anything, look at the data. **Exploratory data analysis (EDA)** means summarising and plotting data to understand it.

```
import pandas as pd

df = pd.read_csv("houses.csv")
print(df.shape)           # rows and columns
print(df.head())          # first five rows
print(df.info())          # column types and missing counts
print(df.describe())       # mean, std, min, max, quartiles
```

Useful questions to ask:

- How many rows and columns are there?
- Which columns have missing values, and how many?
- What are the ranges of numerical features? Are there impossible values, such as a house with -3 bedrooms?
- How are the labels distributed? If 99 percent of transactions are genuine, a model that always says "genuine" is 99 percent accurate but useless.
- Which features seem related to the target?

Plots help enormously: **histograms** show distributions, **box plots** reveal outliers, **scatter plots** show relationships between two variables, and a **correlation heatmap** summarises how numerical features move together.

Handling Missing Values

Real data has gaps: a customer skipped a survey question, a sensor went offline, a field was added to a form only last year.

Options:

1. **Remove rows** with missing values. Simple, but wasteful if many rows are affected, and it can bias the data.
2. **Remove columns** that are mostly empty.
3. **Impute** (fill in) values:
 - numerical: the **mean** or, better when there are outliers, the **median**;
 - categorical: the **most frequent** value, or a new category such as "Unknown";
 - model-based: predict the missing value from other features.
4. **Add an indicator** column, such as `income_missing = 1`, because missingness itself can carry information.

```
df["income"] = df["income"].fillna(df["income"].median())
df["city"] = df["city"].fillna("Unknown")
```

Key idea: Compute imputation values (such as the median) from the **training set only**, then apply the same values to the validation and test sets. Otherwise information from the test set leaks into training.

Outliers

An **outlier** is a value far from the rest. Some are errors, such as a typing mistake that turns age 25 into 250. Others are real but rare, such as a genuine billionaire in an income dataset.

A common rule flags values outside 1.5 times the **interquartile range (IQR)**: below $Q1 - 1.5 \times IQR$ or above $Q3 + 1.5 \times IQR$.

Depending on the cause, you can correct errors, remove them, cap extreme values (**clipping**), or transform the feature, for example by taking a logarithm to shrink large values.

Encoding Categorical Features

Most algorithms need numbers, not words.

- **Label (ordinal) encoding:** map categories to integers.
Good for ordered categories: small = 0, medium = 1, large = 2.
- **One-hot encoding:** create one binary column per category.
City = Pune becomes `city_Pune = 1`, `city_Mumbai = 0`, `city_Delhi = 0`. Use it for nominal categories, because integer codes would suggest a false order (Delhi > Mumbai?).


```
df = pd.get_dummies(df, columns=["city"])
```

With hundreds of categories, one-hot encoding creates too many columns. Alternatives include grouping rare categories into "Other", or **target encoding**, which replaces each category with the average target value for that category (computed carefully to avoid leakage).

Feature Scaling

Features often have very different ranges: area might be 500 to 5,000 square feet, while bedrooms range from 1 to 5.

Algorithms that use distances (k-nearest neighbours, k-means, SVMs) or gradient descent (linear models, neural networks) work much better when features are on similar scales.

- **Min-max normalisation** rescales to the range 0 to 1: $x' = (x - \min) / (\max - \min)$.
- **Standardisation** (z-score) gives mean 0 and standard deviation 1: $x' = (x - \text{mean}) / \text{std}$.

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)    # learn mean
                                                    # and std from training data
X_test_scaled = scaler.transform(X_test)         # reuse them
                                                    # on test data
```

Tree-based models such as decision trees and random forests do not need scaling, because they split on one feature at a time.

Feature Engineering

Feature engineering means creating new features that make patterns easier for the model to find. It is where domain knowledge pays off.

- From a date, extract day of week, month, or whether it is a holiday.
- From height and weight, compute BMI.
- From a transaction history, compute "number of purchases in the last 30 days".
- From text, count words or detect keywords.
- Combine features: price per square foot = price / area.

Key idea: A good feature can improve a model more than switching to a more advanced algorithm.

Feature Selection

More features are not always better. Irrelevant features add noise and increase the risk of overfitting. Feature selection methods include removing features with almost no variation, removing one of two highly correlated features, and keeping the features a model ranks as most important.

Splitting the Data

We divide the dataset into separate parts:

- **Training set** (often 60 to 80 percent): used to fit the model.
- **Validation set** (10 to 20 percent): used to compare models and tune hyperparameters.
- **Test set** (10 to 20 percent): used **once**, at the very end, to estimate real-world performance.

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)
```

`stratify=y` keeps the proportion of each class the same in every split, which matters for imbalanced data. For **time series**, never shuffle: train on the past and test on the future, or the model will "see" the future.

Data Leakage

Data leakage happens when information that would not be available at prediction time sneaks into training. The model then looks brilliant in testing and fails in real use.

Examples:

- Scaling or imputing using statistics from the whole dataset, including test rows.
- Including a feature that is recorded *after* the outcome, such as "refund issued" when predicting whether an order will be returned.

- Having the same customer's records in both training and test sets.

The cure is to split first, then fit every preprocessing step on the training data only. scikit-learn **pipelines** make this automatic.

```
from sklearn.pipeline import make_pipeline
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression

model = make_pipeline(SimpleImputer(strategy="median"),
                      StandardScaler(), LogisticRegression())
model.fit(X_train, y_train)
```

Imbalanced Data

When one class is rare, as with fraud or disease, models tend to ignore it. Remedies include:

- **Oversampling** the minority class, for example with SMOTE, which creates synthetic examples;
- **Undersampling** the majority class;
- using **class weights** so mistakes on the rare class cost more;
- evaluating with precision, recall and F1 instead of accuracy (Chapter 5).

Summary

- Know your feature types: numerical (continuous or discrete) and categorical (nominal or ordinal).

- Explore data before modelling: shapes, types, missing values, ranges and label balance.
- Handle missing values by removing, imputing or flagging them; deal with outliers deliberately.
- One-hot encode nominal categories; scale features for distance- and gradient-based models.
- Engineer informative features and remove useless ones.
- Split into training, validation and test sets, and prevent leakage by fitting preprocessing on training data only.

Practice Questions

1. Classify these features: pin code, temperature, T-shirt size, number of siblings.
2. Why is the median often better than the mean for imputing income?
3. Why is label encoding a bad idea for the feature "city"?
4. Give two examples of data leakage and explain how to prevent each.
5. Which of these models need feature scaling: k-nearest neighbours, decision tree, logistic regression?

Linear Regression

Linear regression is the "hello world" of machine learning. It predicts a number by drawing the best straight line (or flat surface) through the data. It is simple, fast and easy to interpret, and it introduces three ideas that power almost every other model: a **hypothesis**, a **cost function** and **gradient descent**.

The Problem

Suppose we want to predict the price of a flat from its area. We have data on past sales:

Area (sq ft)	Price (lakh)
600	45
850	62
1,000	70
1,200	86
1,500	104

Plotting these points shows a clear upward trend. Linear regression finds the line that fits them best, so we can predict the price of a 1,100 sq ft flat.

The Hypothesis

With one feature, the model is the equation of a line:

$$y_hat = w * x + b$$

- x is the input feature (area),
- y_hat (read "y hat") is the predicted price,
- w is the **weight** or slope: how much the price rises for each extra square foot,
- b is the **bias** or intercept: the predicted price when x is 0.

With many features (area, bedrooms, age of building, distance to station), the model becomes

$$y_hat = w_1*x_1 + w_2*x_2 + \dots + w_n*x_n + b$$

This is **multiple linear regression**. Each weight tells us how much the prediction changes when that feature increases by one unit, keeping the others fixed.

Measuring Error: The Cost Function

For each example, the **residual** (error) is the difference between the actual value and the prediction: $y - y_hat$. We need a single number that summarises how bad the line is overall.

The standard choice is the **mean squared error (MSE)**:

$$MSE = (1/n) * \text{sum over all examples of } (y - y_hat)^2$$

Squaring does two useful things: it makes every error positive, and it punishes large errors much more than small ones.

Training means finding the values of w and b that make the MSE as small as possible.

Key idea: A model is only as good as the thing it is trained to minimise. The cost function defines what "good" means for the learning algorithm.

Finding the Best Line

The Normal Equation

For linear regression there is an exact formula, called the **normal equation**, which gives the best weights in one step using matrix algebra. It works well for small and medium problems but becomes slow when there are very many features, because it requires inverting a large matrix.

Gradient Descent

Gradient descent is a general method used by almost every modern ML model, including huge neural networks.

Imagine standing on a foggy hillside and wanting to reach the valley. You cannot see the bottom, but you can feel the slope under your feet. So you take a step downhill, feel the slope again, and repeat.

1. Start with some values of w and b (often zeros or small random numbers).

2. Compute the **gradient**: the direction in which the cost increases fastest.
3. Take a small step in the **opposite** direction.
4. Repeat until the cost stops decreasing.

```
w = w - learning_rate * (d cost / d w)
b = b - learning_rate * (d cost / d b)
```

For MSE, the gradients are simple averages over the data:

```
d cost / d w = (2/n) * sum( (y_hat - y) * x )
d cost / d b = (2/n) * sum( (y_hat - y) )
```

The Learning Rate

The **learning rate** controls the step size.

- **Too small**: training is painfully slow.
- **Too large**: steps overshoot the valley, and the cost may bounce around or even grow.
- **Just right**: the cost falls steadily and levels off.

Plotting cost against the number of iterations (a **learning curve**) is the easiest way to spot a bad learning rate.

Variants

- **Batch gradient descent** uses all examples for every step. Stable but slow for big data.

- **Stochastic gradient descent (SGD)** uses one random example per step. Fast and noisy.
- **Mini-batch gradient descent** uses small batches, such as 32 or 64 examples. It is the usual compromise and the standard in deep learning.

Linear Regression in Code

```
import numpy as np

x = np.array([600, 850, 1000, 1200, 1500], dtype=float)
y = np.array([45, 62, 70, 86, 104], dtype=float)
x = (x - x.mean()) / x.std()          # scale for stable
                                     gradient descent

w, b, lr = 0.0, 0.0, 0.1
for step in range(500):
    y_hat = w * x + b
    dw = 2 * np.mean((y_hat - y) * x)
    db = 2 * np.mean(y_hat - y)
    w, b = w - lr * dw, b - lr * db

print(w, b)
```

In practice you would simply use scikit-learn:

```
from sklearn.linear_model import LinearRegression

model = LinearRegression().fit(X_train, y_train)
print(model.coef_, model.intercept_)
print(model.predict([[1100]]))
```

Evaluating Regression Models

Metric	Meaning	Notes
MAE	Mean absolute error	Average size of errors, in the target's units
MSE	Mean squared error	Punishes large errors heavily
RMSE	Square root of MSE	Back in the target's units
R^2	Fraction of variance explained	1 is perfect; 0 is no better than predicting the mean

An R^2 of 0.85 means the model explains 85 percent of the variation in prices. RMSE of 5 means predictions are typically off by around 5 lakh.

Assumptions of Linear Regression

Linear regression works best when:

1. **Linearity:** the relationship between features and target is roughly linear.
2. **Independence:** errors for different examples are independent.
3. **Constant variance:** errors are about equally spread for small and large predictions.
4. **Normal errors:** residuals are roughly bell-shaped (important for confidence intervals).

5. **Little multicollinearity:** features are not strongly correlated with each other. If area and number of rooms move together, their individual weights become unreliable.

Plotting residuals against predictions is a quick way to check these assumptions. A random cloud is good; a curve or a funnel shape signals trouble.

Polynomial Regression

What if the relationship is curved? We can still use linear regression by adding **polynomial features** such as x^2 and x^3 . The model is still "linear" in its weights, but it can fit curves.

```
from sklearn.preprocessing import PolynomialFeatures
from sklearn.pipeline import make_pipeline

model = make_pipeline(PolynomialFeatures(degree=2),
                      LinearRegression())
```

Beware: a very high degree fits every wiggle in the training data, which is overfitting.

Regularization

Regularization fights overfitting by adding a penalty for large weights to the cost function.

- **Ridge regression (L2):** adds lambda times the sum of squared weights. It shrinks all weights towards zero but rarely makes them exactly zero.

- **Lasso regression (L1):** adds λ times the sum of absolute weights. It can shrink some weights to exactly zero, which performs **feature selection**.
- **Elastic Net:** a mix of both.

The hyperparameter λ (called `alpha` in scikit-learn) controls the strength. Larger values give simpler models. Features must be scaled before regularization, or the penalty treats them unfairly.

Key idea: Regularization trades a little accuracy on the training data for better generalization to new data.

Summary

- Linear regression predicts a number with a weighted sum of features plus a bias.
- Training minimises a cost function, usually mean squared error.
- Gradient descent repeatedly steps opposite the gradient; the learning rate sets the step size.
- Evaluate with MAE, RMSE and R^2 ; check residual plots for broken assumptions.
- Polynomial features let a linear model fit curves.

- Ridge and Lasso add penalties that reduce overfitting; Lasso can also select features.

Practice Questions

1. What do the weight and bias mean in the model $\text{price} = w * \text{area} + b$?
2. Why do we square errors in MSE instead of just adding them?
3. Describe what happens when the learning rate is too large.
4. What is the difference between Ridge and Lasso regression?
5. A model has $R^2 = 0.98$ on training data and 0.60 on test data. What is going on, and what would you try?

Classification and Logistic Regression

Many real questions have yes-or-no answers. Will this customer cancel? Is this email spam? Does this X-ray show pneumonia? These are **classification** problems. Despite its name, **logistic regression** is one of the most widely used classification algorithms. It is fast, interpretable and a strong baseline for almost any classification task.

Why Not Use Linear Regression?

Suppose we label "spam" as 1 and "not spam" as 0 and fit a straight line. Two problems appear immediately.

1. The line produces values like -0.4 or 1.7, which make no sense as answers or as probabilities.
2. A few extreme examples can tilt the whole line and move the decision boundary in the wrong place.

We need a model whose output always stays between 0 and 1, so we can read it as a **probability**.

The Sigmoid Function

The **sigmoid** (or logistic) function squashes any number into the range 0 to 1:

$$\text{sigmoid}(z) = 1 / (1 + e^{(-z)})$$

- When z is large and positive, $\text{sigmoid}(z)$ is close to 1.
- When z is large and negative, $\text{sigmoid}(z)$ is close to 0.
- When $z = 0$, $\text{sigmoid}(z) = 0.5$.

It draws a smooth S-shaped curve.

The Logistic Regression Model

Logistic regression first computes a linear score, exactly like linear regression, and then passes it through the sigmoid:

$$z = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$$
$$p = \text{sigmoid}(z) \quad \# \text{ probability that } y = 1$$

To turn the probability into a class, we apply a **threshold**, usually 0.5:

predict 1 if $p \geq 0.5$, otherwise predict 0

Key idea: Logistic regression outputs a probability, not just a label. That lets you choose the threshold to suit the problem. A cancer screening test might flag patients at $p \geq 0.2$, because missing a case is worse than an extra check-up.

The Decision Boundary

Because $p \geq 0.5$ exactly when $z \geq 0$, the boundary between the classes is the set of points where $w_1x_1 + \dots + b = 0$. With two features that is a straight line; with more features it is a flat surface (a hyperplane). Logistic regression is therefore a **linear classifier**. Adding polynomial features lets it draw curved boundaries.

Interpreting the Weights

The quantity z is the **log-odds**: $\log(p / (1 - p))$. Each weight says how much the log-odds change when its feature rises by one unit. A positive weight pushes towards class 1 and a negative weight towards class 0. This interpretability makes logistic regression popular in medicine, finance and anywhere decisions must be explained.

The Cost Function: Log Loss

MSE works poorly with the sigmoid, so logistic regression uses **binary cross-entropy**, also called **log loss**:

$$\text{loss} = -[y * \log(p) + (1 - y) * \log(1 - p)]$$

Consider what it does:

- If the true label is 1 and the model says $p = 0.99$, the loss is tiny.
- If the true label is 1 and the model says $p = 0.01$, the loss is huge.

Confident wrong answers are punished severely, which pushes the model towards well-calibrated probabilities. The weights are found with gradient descent, just as in Chapter 3, and log loss has a single minimum, so training is reliable.

Logistic Regression in Code

```
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline

model = make_pipeline(StandardScaler(),
                      LogisticRegression(C=1.0))
model.fit(X_train, y_train)

probs = model.predict_proba(X_test)[: , 1]    # probability of
class 1
preds = (probs >= 0.5).astype(int)
```

In scikit-learn, c is the *inverse* of regularization strength: a smaller C means stronger regularization and a simpler model.

Multi-Class Classification

What if there are more than two classes, such as classifying handwritten digits 0 to 9?

One-vs-Rest

Train one binary classifier per class: "is it a 3 or not?", "is it a 7 or not?", and so on. For a new example, pick the class whose classifier is most confident.

Softmax Regression

Softmax regression (multinomial logistic regression) handles all classes at once. It computes a score for each class and converts the scores into probabilities that add up to 1:

$$p(\text{class } k) = e^{(z_k)} / \sum \text{over all classes } j \text{ of } e^{(z_j)}$$

The loss is **categorical cross-entropy**. Softmax also appears as the final layer of most neural network classifiers.

Setting	Output	Loss
Binary classification	Sigmoid, one probability	Binary cross-entropy
Multi-class (one label)	Softmax, probabilities sum to 1	Categorical cross-entropy
Multi-label (several labels)	One sigmoid per label	Binary cross-entropy per label

Multi-label problems are those where an example can belong to several classes at once, such as a movie that is both "comedy" and "romance".

Choosing the Threshold

The default threshold of 0.5 is not always right. Raising it makes the model more cautious about predicting class 1: fewer false alarms, but more missed positives. Lowering it catches more positives at the cost of more false alarms. Chapter 5 introduces precision, recall and the ROC curve, which help you pick a threshold on purpose.

Other Common Classifiers

Logistic regression is a baseline. Other classifiers you will meet in this book include:

- **k-nearest neighbours:** classify by the majority vote of the closest training examples (Chapter 7).
- **Naive Bayes:** use Bayes' theorem with a simplifying independence assumption. It is very fast and works surprisingly well for text such as spam filtering.
- **Decision trees and random forests** (Chapter 6).
- **Support vector machines** (Chapter 7).
- **Neural networks** (Chapter 9).

A Quick Look at Naive Bayes

Naive Bayes predicts the class with the highest probability given the features, using

$$P(\text{class} \mid \text{features}) \text{ is proportional to } P(\text{class}) * P(\text{feature1} \mid \text{class}) * P(\text{feature2} \mid \text{class}) * \dots$$

It is "naive" because it assumes features are independent given the class, which is rarely exactly true. For spam, it asks: how common is spam overall, and how often do words like "free" and "winner" appear in spam compared with normal mail?

Strengths and Weaknesses of Logistic Regression

Strengths:

- Fast to train, even on large datasets.
- Produces probabilities.
- Weights are interpretable.
- Less prone to overfitting than complex models, especially with regularization.

Weaknesses:

- Draws only linear boundaries unless you engineer features.
- Struggles with complex interactions between features.
- Sensitive to strongly correlated features and to unscaled data.

Summary

- Classification predicts categories; logistic regression is a linear classifier that outputs probabilities.
- The sigmoid squashes a linear score into a probability; a threshold turns it into a class.
- Weights change the log-odds and can be interpreted.
- Training minimises log loss (binary cross-entropy) with gradient descent.

- Multi-class problems use one-vs-rest or softmax with categorical cross-entropy.
- The threshold can and should be tuned to the costs of different mistakes.

Practice Questions

1. Why is linear regression unsuitable for classification?
2. What is $\text{sigmoid}(z)$? What happens to $\text{sigmoid}(z)$ as z becomes very large?
3. Explain why log loss punishes confident wrong predictions more than MSE would.
4. When would you choose a threshold lower than 0.5? Give a real example.
5. What is the difference between multi-class and multi-label classification?

Evaluating Models

How do you know if a model is any good? "It got 95 percent right" sounds impressive, until you learn that 95 percent of the examples belonged to one class. Choosing the right evaluation method is as important as choosing the right algorithm. This chapter covers validation strategies, the metrics used for classification and regression, and the bias-variance trade-off.

Why Training Accuracy Lies

A model evaluated on the same data it was trained on can simply memorise the answers. A decision tree grown without limits often reaches 100 percent training accuracy and then does poorly on new data. **Always evaluate on data the model did not see during training.**

Validation Strategies

Hold-Out Validation

Split the data once into training, validation and test sets (Chapter 2). It is simple and fast, but with small datasets the score depends heavily on which examples happen to land in which split.

K-Fold Cross-Validation

In **k-fold cross-validation**, the training data is split into k equal parts (folds), commonly 5 or 10.

1. Train on $k - 1$ folds and validate on the remaining fold.
2. Repeat k times, so each fold is the validation fold once.
3. Average the k scores.

```
from sklearn.model_selection import cross_val_score

scores = cross_val_score(model, X_train, y_train, cv=5,
                          scoring="f1")
print(scores.mean(), scores.std())
```

Cross-validation uses data efficiently and shows how stable the score is. Its cost is training the model k times.

Variants:

- **Stratified k-fold** keeps class proportions equal in every fold. It is the default for classification.
- **Leave-one-out** uses $k = \text{number of examples}$. Thorough but very slow.
- **Time-series split** always trains on earlier data and validates on later data.

Key idea: Use cross-validation to choose models and hyperparameters. Use the test set exactly once, at the end, to report final performance.

The Confusion Matrix

For a binary classifier, every prediction falls into one of four boxes.

	Predicted positive	Predicted negative
Actually positive	True positive (TP)	False negative (FN)
Actually negative	False positive (FP)	True negative (TN)

Consider a disease test on 1,000 people, 50 of whom are sick. The model flags 60 people; 40 of them are truly sick.

- TP = 40 (sick and flagged)
- FN = 10 (sick but missed)
- FP = 20 (healthy but flagged)
- TN = 930 (healthy and not flagged)

A **false positive** is a false alarm (Type I error). A **false negative** is a miss (Type II error). Which is worse depends on the problem.

Classification Metrics

Accuracy

$$\text{accuracy} = (\text{TP} + \text{TN}) / \text{total} = (40 + 930) / 1000 = 97\%$$

Accuracy is misleading for imbalanced data. A model that says "healthy" for everyone scores 95 percent here while finding no sick patients at all.

Precision

Of everything the model flagged as positive, how much was really positive?

$$\text{precision} = \text{TP} / (\text{TP} + \text{FP}) = 40 / 60 = 0.67$$

High precision means few false alarms. It matters when false positives are costly, such as marking genuine emails as spam.

Recall (Sensitivity)

Of all the real positives, how many did the model find?

$$\text{recall} = \text{TP} / (\text{TP} + \text{FN}) = 40 / 50 = 0.80$$

High recall means few misses. It matters when false negatives are costly, such as missing a disease or a fraudulent transaction.

The Precision-Recall Trade-off

Lowering the classification threshold flags more examples: recall rises but precision usually falls. Raising the threshold does the opposite. You cannot usually maximise both, so decide which mistake is more expensive.

F1 Score

The **F1 score** is the harmonic mean of precision and recall:

$$\text{F1} = 2 * \text{precision} * \text{recall} / (\text{precision} + \text{recall}) = 2 * 0.67 * 0.80 / 1.47 = 0.73$$

The harmonic mean is low if *either* value is low, so F1 rewards balanced models. It is a good single number for imbalanced

problems.

Specificity

$$\text{specificity} = \text{TN} / (\text{TN} + \text{FP}) = 930 / 950 = 0.98$$

Specificity is the recall of the negative class: how many healthy people were correctly cleared.

ROC Curves and AUC

A **ROC curve** plots the **true positive rate** (recall) against the **false positive rate** ($\text{FP} / (\text{FP} + \text{TN})$) for every possible threshold.

- A perfect model reaches the top-left corner: 100 percent recall with no false positives.
- A random guesser follows the diagonal line.

The **area under the curve (AUC)** summarises the whole curve in one number between 0.5 (random) and 1.0 (perfect). AUC has a neat interpretation: it is the probability that the model gives a randomly chosen positive a higher score than a randomly chosen negative.

For heavily imbalanced data, the **precision-recall curve** and its area are often more informative than ROC, because ROC can look good even when precision is poor.

Metrics for Multi-Class Problems

Precision, recall and F1 are computed per class and then averaged.

- **Macro average:** the plain average across classes. Every class counts equally, so rare classes matter.
- **Weighted average:** weights each class by how many examples it has.
- **Micro average:** pools all TP, FP and FN counts first.

A multi-class confusion matrix shows which classes are confused with which, such as a digit model mixing up 4 and 9.

Regression Metrics Revisited

Metric	Good for
MAE	Easy explanation; robust to outliers
RMSE	Penalising large errors
R^2	Comparing against the "predict the mean" baseline
MAPE	Errors as percentages, when the target is never near zero

The Bias-Variance Trade-off

A model's error on new data comes from three sources.

- **Bias:** error from wrong assumptions. A straight line fitted to a curve has high bias. High bias causes **underfitting**.

- **Variance:** error from being too sensitive to the particular training set. A very deep tree changes wildly if a few examples change. High variance causes **overfitting**.
- **Irreducible noise:** randomness in the data that no model can remove.

Simple models tend to have high bias and low variance; complex models have low bias and high variance. The best model balances the two.

Key idea: Diagnose with training and validation scores. Both low: high bias, so use a more powerful model or better features. Training high but validation low: high variance, so get more data, simplify or regularize.

Learning Curves

A **learning curve** plots training and validation scores against training-set size.

- If both curves are low and close together, more data will not help much; the model is too simple.
- If there is a big gap between them that narrows as data grows, more data will help.

Hyperparameter Tuning

Hyperparameters, such as tree depth or regularization strength, are chosen using validation scores.

- **Grid search** tries every combination from a list of values.
- **Random search** tries random combinations. It is often more efficient when only a few hyperparameters really matter.
- **Bayesian optimisation** uses past results to pick promising values next.

```
from sklearn.model_selection import GridSearchCV

grid = GridSearchCV(model, {"logisticregression__C": [0.01,
    0.1, 1, 10]}, cv=5, scoring="f1")
grid.fit(X_train, y_train)
print(grid.best_params_, grid.best_score_)
```

Summary

- Never judge a model on its training data; use hold-out sets or k-fold cross-validation.
- The confusion matrix counts TP, FP, FN and TN.
- Accuracy misleads on imbalanced data; use precision, recall, F1 and specificity.
- ROC-AUC measures ranking quality across thresholds; precision-recall curves suit rare positives.
- Bias causes underfitting and variance causes overfitting; learning curves help diagnose which.

- Tune hyperparameters on validation data with grid, random or Bayesian search.

Practice Questions

1. A fraud model has 99.5 percent accuracy on data where 0.5 percent of transactions are fraud. Why might it still be useless?
2. From $TP = 30$, $FP = 10$, $FN = 20$, $TN = 940$, compute precision, recall and F1.
3. For a cancer screening test, would you prioritise precision or recall? Why?
4. Explain 5-fold cross-validation step by step.
5. Training accuracy is 99 percent and validation accuracy is 70 percent. Name three things you could try.

Decision Trees and Ensembles

Decision trees make predictions the way people often make decisions: by asking a sequence of simple questions. They are easy to understand and work with messy, mixed data. On their own they tend to overfit, but combined into **ensembles** such as random forests and gradient boosting, they become some of the most accurate models for tabular data.

How a Decision Tree Works

Imagine deciding whether a bank should approve a loan.

```
Is income > 50,000?  
|-- No -> Is there a guarantor?  
|         |-- No -> Reject  
|         |-- Yes -> Approve  
|-- Yes -> Credit score > 700?  
|         |-- No -> Reject  
|         |-- Yes -> Approve
```

A decision tree has:

- a **root node**: the first question,
- **internal nodes**: further questions, each testing one feature,
- **branches**: the answers,
- **leaf nodes**: the final predictions.

To classify a new applicant, start at the root and follow the answers down to a leaf. For regression trees, a leaf predicts a number, usually the average target value of the training examples that reached it.

Learning a Tree

The algorithm builds the tree greedily from the top down:

1. Consider every feature and every possible split point.
2. Choose the split that best separates the classes, making the child nodes as **pure** as possible.
3. Repeat recursively for each child.
4. Stop when a node is pure, too small, or a depth limit is reached.

"Pure" means a node contains examples of only one class. We need a way to measure impurity.

Gini Impurity

$$\text{Gini} = 1 - \sum \text{over classes of } (p_k)^2$$

where p_k is the fraction of examples in the node belonging to class k .

- A pure node (all one class) has $\text{Gini} = 0$.
- A 50/50 node in a binary problem has $\text{Gini} = 1 - (0.25 + 0.25) = 0.5$, the worst case.

Entropy and Information Gain

$$\text{Entropy} = - \sum \text{over classes of } p_k * \log_2(p_k)$$

Entropy is 0 for a pure node and 1 for a 50/50 binary node.

Information gain is the drop in entropy achieved by a split: the entropy of the parent minus the weighted average entropy of the children. The algorithm picks the split with the highest gain.

In practice Gini and entropy give very similar trees. Gini is slightly faster to compute and is the default in scikit-learn.

A Tiny Worked Example

A node contains 10 loan applications: 6 approved and 4 rejected. Its Gini is $1 - (0.6^2 + 0.4^2) = 1 - 0.52 = 0.48$. Splitting on "income > 50,000" gives a left child with 4 rejected and 1 approved and a right child with 5 approved. The right child is pure (Gini 0). The left child has Gini $1 - (0.8^2 + 0.2^2) = 0.32$. The weighted impurity is $0.5 \times 0.32 + 0.5 \times 0 = 0.16$, down from 0.48, which makes this a good split.

Splitting Regression Trees

For regression, the tree chooses splits that minimise the **variance** (or MSE) of the target inside each child.

Overfitting and Pruning

A tree allowed to grow without limits keeps splitting until every leaf is pure. It will often end up with one leaf per training example, perfectly memorising the data, noise included.

Ways to control this:

- **Pre-pruning** (early stopping): limit `max_depth`, require `min_samples_split` OR `min_samples_leaf`, or cap the number of leaves.
- **Post-pruning**: grow a full tree, then remove branches that do not improve validation performance. **Cost-complexity pruning** (`ccp_alpha` in scikit-learn) does this systematically.

```
from sklearn.tree import DecisionTreeClassifier, export_text

tree = DecisionTreeClassifier(max_depth=4,
                             min_samples_leaf=20, random_state=0)
tree.fit(X_train, y_train)
print(export_text(tree, feature_names=list(X_train.columns)))
```

Strengths and Weaknesses of Trees

Strengths:

- Easy to visualise and explain.
- Handle numerical and categorical features, and need no feature scaling.
- Capture non-linear relationships and feature interactions automatically.

Weaknesses:

- Overfit easily.

- **Unstable:** a small change in the data can produce a completely different tree.
- Make boxy, step-shaped predictions.

Ensemble Learning

An **ensemble** combines many models to get a better one. The idea is the "wisdom of crowds": if many reasonably good models make *different* mistakes, their combined vote is more accurate than any one of them.

Key idea: Ensembles work best when the individual models are diverse. Averaging many copies of the same model achieves nothing.

Bagging

Bagging (bootstrap aggregating) trains many models on different **bootstrap samples** of the training data: random samples of the same size drawn *with replacement*, so some examples appear twice and others not at all. Predictions are combined by majority vote (classification) or averaging (regression).

Bagging mainly **reduces variance**, which makes it ideal for unstable, high-variance models like deep trees.

Random Forests

A **random forest** is bagging with decision trees plus one extra twist: at each split, a tree may only consider a **random subset of the features**. This stops every tree from splitting on the same strong feature at the top, making the trees more diverse.

```
from sklearn.ensemble import RandomForestClassifier

forest = RandomForestClassifier(n_estimators=300,
                               max_features="sqrt", n_jobs=-1, random_state=0)
forest.fit(X_train, y_train)
print(forest.score(X_test, y_test))
```

Useful facts:

- More trees rarely hurt; they just cost time.
- **Out-of-bag (OOB) evaluation:** each tree never saw roughly a third of the data, so those examples provide a free validation score.
- **Feature importance:** forests report how much each feature reduced impurity across all trees. It is a helpful, though imperfect, guide to what the model relies on.

Random forests are an excellent default: accurate, robust and hard to break.

Boosting

Boosting builds models **sequentially**, with each new model focusing on the mistakes of the ones before it. It mainly **reduces bias**, turning many weak learners (often shallow trees) into a strong one.

- **AdaBoost** increases the weight of misclassified examples so the next model pays more attention to them. The final prediction is a weighted vote.
- **Gradient boosting** fits each new tree to the **residual errors** (more precisely, the gradient of the loss) of the current ensemble, and adds it with a small **learning rate**.

```
prediction = tree1 + lr * tree2 + lr * tree3 + ...
```

Popular, highly optimised implementations include **XGBoost**, **LightGBM** and **CatBoost**. They dominate machine learning competitions on tabular data.

Key hyperparameters for gradient boosting:

- `n_estimators`: number of trees.
- `learning_rate`: how much each tree contributes. Smaller values need more trees but generalise better.
- `max_depth`: depth of each tree, usually small (3 to 8).
- `subsample`: fraction of rows used per tree, which adds randomness and reduces overfitting.

Boosting can overfit if it runs too long, so use **early stopping** on a validation set.

Stacking

Stacking trains several different models (say a random forest, logistic regression and gradient boosting), then trains a final **meta-model** on their predictions to learn how best to combine them.

Comparing the Approaches

Method	Trains models	Mainly reduces	Typical base model
Bagging / random forest	In parallel, independently	Variance	Deep trees
Boosting	Sequentially, fixing errors	Bias	Shallow trees
Stacking	Different model types	Both	Anything

Summary

- Decision trees split data with simple feature tests, choosing splits that reduce Gini impurity or entropy.
- Unrestricted trees overfit; control depth and leaf size or prune them.
- Ensembles combine diverse models to beat any single one.

- Bagging and random forests average many trees trained on bootstrap samples, reducing variance.
- Boosting adds trees one at a time to fix earlier errors, reducing bias; tune the learning rate and use early stopping.
- For tabular data, random forests and gradient boosting are strong first choices.

Practice Questions

1. Compute the Gini impurity of a node containing 8 examples of class A and 2 of class B.
2. Why does an unpruned decision tree overfit?
3. What extra randomness does a random forest add compared with plain bagging, and why does it help?
4. Explain the difference between bagging and boosting in terms of bias and variance.
5. What does a small learning rate do in gradient boosting?

Nearest Neighbours and Support Vector Machines

This chapter covers two classic algorithms with very different personalities. **k-nearest neighbours** is almost laughably simple: to classify something, look at the most similar examples you have already seen. **Support vector machines** are mathematically elegant: they find the widest possible "street" separating two classes. Both rely on the idea of distance or geometry in feature space, so both care a lot about feature scaling.

Feature Space

Think of every example as a point in space, with one axis per feature. A student described by hours studied and hours slept is a point in 2-D space; a house with ten features is a point in 10-D space. Similar examples are points that lie close together. Classification then becomes a question of geometry: which region of the space does a new point fall into?

k-Nearest Neighbours (kNN)

The Algorithm

To classify a new point:

1. Compute its distance to every training example.

2. Pick the **k** closest examples, its "neighbours".
3. **Classification**: predict the most common class among the neighbours.
4. **Regression**: predict the average of the neighbours' target values.

That is the whole algorithm. There is no real training step; the model simply stores the data. kNN is therefore called a **lazy learner** or an **instance-based** method.

Measuring Distance

The most common choice is **Euclidean distance**, the straight-line distance:

$$d(a, b) = \sqrt{(a_1 - b_1)^2 + (a_2 - b_2)^2 + \dots + (a_n - b_n)^2}$$

Other options:

- **Manhattan distance**: the sum of absolute differences, like walking city blocks: $|a_1 - b_1| + |a_2 - b_2| + \dots$
- **Cosine similarity**: compares the *direction* of two vectors rather than their size. It is popular for text, where documents of different lengths should still match if they use words in similar proportions.

Key idea: kNN is only as good as its distance measure. If one feature ranges from 0 to 100,000 (income) and another from 0 to 5 (rating), income completely dominates the distance. Always scale features before using kNN.

Choosing k

- **Small k** (such as 1): the model follows every quirk of the training data. It has low bias and high variance and is sensitive to noise.
- **Large k:** predictions become smoother, but eventually the model ignores local structure. High bias, low variance.

Choose k with cross-validation. For binary classification, an odd k avoids ties. A useful refinement is **distance-weighted voting**, in which nearer neighbours count more than distant ones.

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline

knn = make_pipeline(StandardScaler(),
                    KNeighborsClassifier(n_neighbors=7, weights="distance"))
knn.fit(X_train, y_train)
```

Strengths and Weaknesses of kNN

Strengths:

- Very simple to understand and implement.

- Makes no assumptions about the shape of the decision boundary.
- New data can be added instantly without retraining.

Weaknesses:

- **Slow predictions:** every prediction compares against all training points. Data structures such as KD-trees and ball trees, or approximate nearest-neighbour search, help.
- Needs lots of memory to store the dataset.
- Sensitive to irrelevant features and unscaled data.
- Suffers badly from the curse of dimensionality.

The Curse of Dimensionality

As the number of features grows, space becomes enormous and data becomes sparse. In very high dimensions, almost all points end up roughly the same distance from each other, so "nearest" loses its meaning. Distance-based methods then perform poorly unless you reduce the number of dimensions first (Chapter 8) or select relevant features.

Support Vector Machines (SVM)

The Maximum Margin Idea

Suppose two classes can be separated by a straight line. Usually many lines would work. Which is best?

An SVM chooses the line (in higher dimensions, a **hyperplane**) that has the **largest margin**: the greatest distance to the nearest points of either class. Picture the widest possible street between the two classes, with the decision boundary running down the middle.

The training points that lie on the edges of the street are called **support vectors**. They alone determine the boundary; moving other points (without crossing the street) changes nothing. This makes SVMs robust and memory-efficient at prediction time.

Key idea: A wide margin means the classifier is not "cutting it close" to any training example, which tends to generalise better to new data.

Soft Margins and the C Parameter

Real data is rarely perfectly separable, and outliers can force a very narrow street. A **soft-margin SVM** allows some points to fall inside the street or even on the wrong side, at a cost.

The hyperparameter **C** controls the trade-off:

- **Large C:** mistakes are expensive, so the model tries hard to classify every training point correctly. The margin is narrower and there is more risk of overfitting.
- **Small C:** mistakes are cheap, so the margin is wider and the model is simpler and more tolerant of outliers.

The Kernel Trick

What if the classes cannot be separated by any straight line, say one class forms a ring around the other? The trick is to map the data into a higher-dimensional space where a flat separator *does* exist. Adding the feature $x^2 + y^2$, for example, turns the ring problem into a simple threshold.

Computing such mappings explicitly can be very expensive. The **kernel trick** lets the SVM compute similarities *as if* the data had been mapped, without ever building the new features.

Common kernels:

Kernel	Boundary shape	Notes
Linear	Straight line / hyperplane	Fast; good for text and many features
Polynomial	Curved, of a set degree	Degree is a hyperparameter
RBF (Gaussian)	Flexible, smooth regions	The most popular default
Sigmoid	Similar to a neural network	Rarely used

The RBF kernel has a hyperparameter **gamma**. A large gamma makes each training point's influence very local, giving wiggly boundaries that can overfit. A small gamma makes boundaries smoother. Tune C and gamma together with a grid search.

```
from sklearn.svm import SVC

svm = make_pipeline(StandardScaler(), SVC(kernel="rbf",
      C=1.0, gamma="scale", probability=True))
svm.fit(X_train, y_train)
```

SVMs for Regression

Support vector regression (SVR) flips the idea: it fits a tube of width epsilon around the data and ignores errors that fall inside the tube, penalising only points outside it.

Strengths and Weaknesses of SVMs

Strengths:

- Effective in high-dimensional spaces, even when features outnumber examples.
- Memory-efficient, because only support vectors matter.
- Flexible through kernels.

Weaknesses:

- Training is slow on very large datasets (hundreds of thousands of rows or more).
- Sensitive to feature scaling and to the choice of C and gamma.
- Do not directly produce probabilities; extra calibration is needed.

- Harder to interpret than trees or linear models.

kNN vs SVM at a Glance

	kNN	SVM
Training cost	None (stores data)	Can be high
Prediction cost	High (compares with all data)	Low (support vectors only)
Key hyperparameters	k, distance measure	C, kernel, gamma
Needs scaling	Yes	Yes
Handles high dimensions	Poorly	Well

Summary

- Examples are points in feature space; similar examples lie close together.
- kNN predicts from the k closest training points. It is lazy, simple, and very sensitive to scaling and dimensionality.
- Small k overfits; large k underfits. Choose k with cross-validation.
- SVMs find the maximum-margin hyperplane, defined by the support vectors.
- C trades margin width against training errors; kernels such as RBF handle non-linear boundaries.

- Both methods need scaled features and tuned hyperparameters.

Practice Questions

1. Compute the Euclidean and Manhattan distances between (1, 2) and (4, 6).
2. Why must features be scaled before using kNN? Give an example of what goes wrong otherwise.
3. What happens to a kNN classifier as k grows very large?
4. What are support vectors, and why are they important?
5. Explain the effect of increasing C in a soft-margin SVM.

Unsupervised Learning: Clustering and Dimensionality Reduction

So far every dataset has come with answers. Often, though, we have plenty of data and no labels at all: millions of customers, thousands of documents, endless sensor readings.

Unsupervised learning finds structure in such data. This chapter covers the two biggest families: **clustering**, which groups similar examples, and **dimensionality reduction**, which compresses many features into a few.

Clustering

Clustering divides data into groups (clusters) such that examples in the same cluster are similar and examples in different clusters are dissimilar.

Common uses:

- **Customer segmentation:** find groups such as bargain hunters, loyal regulars and occasional big spenders, and market to each differently.
- **Document grouping:** organise news articles by topic.
- **Image compression:** replace thousands of colours with the few cluster centres.

- **Anomaly detection:** points far from every cluster may be suspicious.

k-Means Clustering

k-means is the most popular clustering algorithm.

The Algorithm

1. Choose the number of clusters, **k**.
2. Place **k centroids** at random (often at **k** random data points).
3. **Assign** each point to its nearest centroid.
4. **Update** each centroid to be the mean of the points assigned to it.
5. Repeat steps 3 and 4 until assignments stop changing.

The algorithm minimises the **within-cluster sum of squares** (also called **inertia**): the total squared distance from each point to its centroid.

```
from sklearn.cluster import KMeans

km = KMeans(n_clusters=4, n_init=10, random_state=0)
labels = km.fit_predict(X_scaled)
print(km.cluster_centers_)
```

Key idea: k-means always converges, but possibly to a poor solution that depends on where the centroids started. Running it several times with different starts (`n_init`) and using the smarter **k-means++** initialisation largely solves this.

Choosing k

- **Elbow method:** run k-means for $k = 1, 2, 3, \dots$ and plot inertia. Inertia always falls as k grows, but the curve usually bends at some point. That "elbow" is a reasonable choice.
- **Silhouette score:** for each point, compare its average distance to its own cluster (a) with its average distance to the nearest other cluster (b). The silhouette is $(b - a) / \max(a, b)$, ranging from -1 to 1 . Higher is better. Choose the k with the highest average silhouette.
- **Domain knowledge:** sometimes the business needs a specific number of segments.

Limitations of k-Means

- You must choose k in advance.
- It assumes clusters are roughly round and of similar size. It fails on long, curved or nested shapes.
- It is sensitive to outliers, which drag centroids towards them.
- Features must be scaled, because it relies on distances.

Hierarchical Clustering

Agglomerative hierarchical clustering builds a tree of clusters from the bottom up:

1. Start with every point as its own cluster.
2. Repeatedly merge the two closest clusters.
3. Continue until only one cluster remains.

The result is drawn as a **dendrogram**, a tree diagram showing the order and distance of merges. Cutting the dendrogram at a chosen height gives any number of clusters, so you do not need to fix k beforehand.

How is the distance between two *clusters* measured? This is the **linkage**:

- **Single linkage:** distance between the closest pair of points. It can create long, chained clusters.
- **Complete linkage:** distance between the farthest pair. It produces compact clusters.
- **Average linkage:** average distance over all pairs.
- **Ward's method:** merge the pair that increases total within-cluster variance the least. It is often the best default.

Hierarchical clustering is intuitive, but it is slow for large datasets, because it compares all pairs of points.

DBSCAN: Density-Based Clustering

DBSCAN defines clusters as dense regions separated by sparse regions. It has two parameters:

- **eps**: the radius of a neighbourhood;
- **min_samples**: how many points must lie within eps for a point to be a **core point**.

Core points that are close together form a cluster; points near a cluster but not dense themselves are **border points**; points in no dense region are labelled **noise**.

Advantages: it finds clusters of **any shape**, it does not need k, and it identifies outliers automatically. Disadvantages: choosing eps can be tricky, and it struggles when clusters have very different densities.

Algorithm	Need k?	Cluster shapes	Handles outliers
k-means	Yes	Round, similar size	Poorly
Hierarchical	No (cut later)	Depends on linkage	Moderately
DBSCAN	No	Any shape	Yes, labels them as noise

Dimensionality Reduction

Datasets can have hundreds or thousands of features.

Dimensionality reduction represents the data with fewer features while keeping as much useful information as possible.

Why bother?

- **Visualisation:** we can only plot 2 or 3 dimensions.
- **Speed:** fewer features mean faster training.
- **Noise reduction:** discarding minor directions can remove noise.
- **Avoiding the curse of dimensionality** for distance-based methods.

Principal Component Analysis (PCA)

PCA finds new axes, called **principal components**, that capture the most variation in the data.

- The **first principal component** is the direction along which the data varies the most.
- The **second** is the direction of greatest remaining variation that is **perpendicular** (uncorrelated) to the first.
- And so on.

We then keep only the first few components. Imagine photographing a 3-D object: a good camera angle keeps most of

its shape in a 2-D picture. PCA finds the best angle mathematically.

Steps

1. Standardise the features (mean 0, standard deviation 1).
2. Compute the covariance matrix of the features.
3. Find its **eigenvectors** (the directions of the components) and **eigenvalues** (how much variance each direction captures).
4. Sort components by eigenvalue and keep the top ones.
5. Project the data onto them.

```
from sklearn.decomposition import PCA

pca = PCA(n_components=0.95)          # keep enough
    components for 95% of the variance
X_reduced = pca.fit_transform(X_scaled)
print(pca.n_components_, pca.explained_variance_ratio_)
```

The **explained variance ratio** tells you what fraction of the total variance each component captures. A **scree plot** of these values helps decide how many components to keep.

Key idea: Principal components are combinations of the original features, so they are harder to interpret. PCA also only captures *linear* structure.

Feature Selection vs Feature Extraction

- **Feature selection** keeps a subset of the original features. They stay interpretable.
- **Feature extraction** (like PCA) creates new features from combinations of the old ones.

Non-Linear Methods for Visualisation

t-SNE and **UMAP** are non-linear techniques designed mainly for visualising high-dimensional data in 2-D. They place similar points close together, often revealing clusters beautifully, for example digits of the same kind forming islands. They are great for exploration, but distances between far-apart clusters in the plot are not meaningful, and they are rarely used as input features for other models.

Evaluating Unsupervised Models

Without labels, evaluation is harder.

- **Internal metrics** such as silhouette score and inertia measure cluster quality from the data alone.
- **External metrics** such as the adjusted Rand index compare clusters with known labels, when some exist.
- **Usefulness** is often the real test: do the customer segments lead to better marketing results?

Summary

- Unsupervised learning finds structure without labels.

- k-means assigns points to the nearest centroid and updates centroids until stable; choose k with the elbow method or silhouette score.
- Hierarchical clustering builds a dendrogram using a linkage rule; DBSCAN finds dense regions of any shape and labels noise.
- Dimensionality reduction helps visualisation, speed and noise reduction.
- PCA finds uncorrelated directions of maximum variance; keep enough components to explain most of the variance.
- t-SNE and UMAP are for visualisation, not general feature engineering.

Practice Questions

1. Walk through two iterations of k-means on a small example of your own.
2. What does the elbow in an elbow plot represent?
3. When would DBSCAN be a better choice than k-means?
4. Why must features be standardised before applying PCA?
5. A PCA shows the first two components explain 92 percent of the variance. What does that mean in plain words?

Neural Networks and Deep Learning

Neural networks power the most visible achievements of modern AI: recognising faces, translating languages, generating images and holding conversations. The core ideas, however, are surprisingly approachable. A neural network is a stack of simple units, each doing a weighted sum followed by a small non-linear function, trained with the same gradient descent you met in Chapter 3.

From Neurons to Perceptrons

Biological neurons receive signals through dendrites, combine them, and fire an output signal if the combined input is strong enough. The artificial **neuron** (or **perceptron**) is a loose mathematical imitation:

```
z = w1*x1 + w2*x2 + ... + wn*xn + b
output = activation(z)
```

Each input has a **weight**, there is a **bias**, and an **activation function** decides the output. A single perceptron with a step activation can learn to separate two classes with a straight line, just like logistic regression. It famously cannot learn the XOR function, which needs a curved boundary. The solution is to stack neurons into layers.

Multi-Layer Networks

A **feed-forward neural network**, also called a **multi-layer perceptron (MLP)**, has:

- an **input layer**, with one value per feature;
- one or more **hidden layers** of neurons, each connected to every neuron in the previous layer;
- an **output layer**, which produces the prediction.

Data flows forward from inputs to outputs. Each hidden layer builds new features out of the previous layer's outputs. In an image model, early layers might detect edges, middle layers shapes, and later layers whole objects. "Deep" learning simply means networks with many hidden layers.

Key idea: The power of neural networks comes from learning features automatically, layer by layer, instead of relying on hand-engineered features.

Activation Functions

Without non-linear activations, a stack of layers would collapse into one big linear function, no more powerful than linear regression. Activations add the non-linearity.

Activation	Formula (idea)	Typical use
Sigmoid	Squashes to 0 to 1	Binary output layer

Activation	Formula (idea)	Typical use
Tanh	Squashes to -1 to 1	Older hidden layers, some recurrent nets
ReLU	$\max(0, z)$	Default for hidden layers
Leaky ReLU	Small slope for negative z	Avoids "dead" ReLU units
Softmax	Probabilities summing to 1	Multi-class output layer

ReLU (rectified linear unit) is the default in hidden layers because it is cheap to compute and helps gradients flow in deep networks. Sigmoid and tanh flatten out for large inputs, which slows learning.

Training: Forward Pass, Loss and Backpropagation

Training repeats four steps for each mini-batch of data.

1. **Forward pass:** feed inputs through the network to get predictions.
2. **Compute the loss:** compare predictions with true labels, using MSE for regression and cross-entropy for classification.
3. **Backward pass (backpropagation):** compute the gradient of the loss with respect to every weight, working backwards from the output layer. It uses the **chain rule** from calculus to reuse intermediate results efficiently.

4. **Update weights:** move each weight a small step against its gradient.

One full pass through the training data is an **epoch**. Networks typically train for many epochs.

Optimizers

Plain gradient descent can be slow or erratic. Improved **optimizers** adjust the steps intelligently:

- **SGD with momentum** keeps a running average of past gradients, like a ball rolling downhill that builds up speed and rolls through small bumps.
- **RMSProp** adapts the step size of each weight based on the size of its recent gradients.
- **Adam** combines momentum and adaptive step sizes. It is the usual default.

A Neural Network in Code

Using the Keras API:

```
from tensorflow import keras

model = keras.Sequential([
    keras.layers.Input(shape=(20,)),
    keras.layers.Dense(64, activation="relu"),
    keras.layers.Dropout(0.3),
    keras.layers.Dense(32, activation="relu"),
    keras.layers.Dense(1, activation="sigmoid"),
])
model.compile(optimizer="adam", loss="binary_crossentropy",
              metrics=["accuracy"])
model.fit(X_train, y_train, epochs=30, batch_size=32,
          validation_split=0.2)
```

PyTorch is the other major framework, popular in research. The ideas are identical; only the style differs.

Fighting Overfitting

Neural networks have many parameters, often millions, so they overfit easily.

- **More data** is the most reliable cure. **Data augmentation** creates extra training examples by, for instance, flipping, rotating or cropping images.
- **Dropout** randomly switches off a fraction of neurons during each training step, so the network cannot rely on any single neuron.
- **Weight decay (L2 regularization)** penalises large weights.

- **Early stopping** halts training when validation loss stops improving.
- **Batch normalization** standardises the inputs to each layer, which speeds up and stabilises training and adds a mild regularizing effect.

Vanishing and Exploding Gradients

In very deep networks, gradients can shrink towards zero as they travel backwards through many layers, so early layers barely learn (**vanishing gradients**), or they can grow uncontrollably (**exploding gradients**). ReLU activations, careful weight initialisation, batch normalization, **residual (skip) connections** and gradient clipping all help.

Specialised Architectures

Convolutional Neural Networks (CNNs)

Images have spatial structure: nearby pixels are related. A **CNN** uses **convolutional layers**, in which small filters (such as 3 x 3 grids of weights) slide across the image, detecting local patterns like edges and textures. The same filter is reused everywhere, which massively reduces the number of parameters. **Pooling layers** shrink the image while keeping the strongest signals. CNNs dominate image classification, object detection and medical imaging.

Recurrent Neural Networks (RNNs)

Text, speech and time series are **sequences**. An **RNN** processes one element at a time while carrying a hidden state, a kind of memory, from step to step. Plain RNNs forget long-range information, so improved versions called **LSTM** and **GRU** add gates that control what to remember and what to forget.

Transformers

Transformers replaced RNNs for most language tasks. Their key mechanism, **self-attention**, lets every word in a sentence look directly at every other word and decide which ones matter for its meaning. Transformers process sequences in parallel, which makes them fast to train on huge datasets. Large language models, modern translation systems and many image models are transformers.

Transfer Learning

Training a large network from scratch needs enormous data and computing power. **Transfer learning** starts from a model already trained on a big dataset, such as an image model trained on millions of photos, and **fine-tunes** it on your smaller dataset. The early layers already know general features like edges and shapes; only the later layers need adapting. This lets small teams build strong models with a few thousand examples.

When to Use Deep Learning

Deep learning shines with **unstructured data** (images, audio, text) and large datasets. For small or medium **tabular** datasets, gradient-boosted trees (Chapter 6) are usually as accurate or better, and faster and easier to tune. Pick the tool that fits the data.

Summary

- A neuron computes a weighted sum plus bias and applies an activation function.
- Stacking layers with non-linear activations lets networks learn complex functions and their own features.
- Training uses forward passes, a loss, backpropagation and an optimizer such as Adam.
- Dropout, weight decay, early stopping, data augmentation and batch normalization fight overfitting.
- CNNs suit images, RNNs and LSTMs suit sequences, and transformers use attention and power modern language models.
- Transfer learning reuses pre-trained networks; for tabular data, tree ensembles often win.

Practice Questions

1. Why does a network need non-linear activation functions?

2. Describe the four steps of one training iteration.
3. What does dropout do, and why does it reduce overfitting?
4. Why are CNNs better suited to images than fully connected networks?
5. Explain transfer learning with an example relevant to your college.

Machine Learning in Practice and Ethics

A model that scores well in a notebook is only the beginning. It has to be deployed, kept healthy as the world changes, and used fairly and responsibly. This final chapter covers how ML systems work in the real world, and the ethical questions every practitioner should ask.

From Notebook to Product

A typical production ML system has several parts beyond the model itself.

1. **Data pipeline:** collects, validates and transforms data regularly.
2. **Training pipeline:** retrains the model on fresh data, evaluates it and stores it with a version number.
3. **Model registry:** keeps track of every model version, its metrics and the data it was trained on.
4. **Serving:** makes predictions available to applications.
5. **Monitoring:** watches inputs, predictions and outcomes over time.

The discipline of running these systems reliably is called **MLOps**, bringing ideas from software engineering (version control, automated testing, continuous deployment) to machine learning.

Ways to Serve Predictions

- **Batch prediction:** compute predictions for everyone at once on a schedule, such as nightly churn scores for all customers. Simple and efficient.
- **Online (real-time) prediction:** a web service returns a prediction on request within milliseconds, such as fraud checks during payment.
- **On-device prediction:** the model runs on the phone or device itself, such as keyboard suggestions. This is fast, works offline and keeps data private, but needs small models.

A minimal online service might look like this:

```
from fastapi import FastAPI
import joblib

app = FastAPI()
model = joblib.load("churn_model.joblib")

@app.post("/predict")
def predict(customer: dict):
    features = [[customer["tenure"],
                    customer["monthly_bill"], customer["complaints"]]]
    return {"churn_probability":
            float(model.predict_proba(features)[0, 1])}
```

Reproducibility

Could you rebuild last month's model exactly? Reproducibility requires:

- versioned **code** (git),
- versioned **data**, or at least a record of exactly which data was used,
- fixed **random seeds**,
- recorded **hyperparameters** and library versions.

Experiment-tracking tools such as MLflow and Weights & Biases record these details automatically.

Monitoring and Drift

The world changes, and models trained on the past slowly go stale.

- **Data drift:** the distribution of inputs changes. A loan model trained before a recession sees very different incomes afterwards.
- **Concept drift:** the relationship between inputs and outputs changes. Fraudsters invent new tricks, so old fraud patterns stop working.

Key idea: Every deployed model needs monitoring. Track input statistics, prediction distributions and, when labels arrive later, real accuracy. Retrain when performance drops.

Explainability

Many decisions must be explained: why was this loan rejected, why was this patient flagged? Some models are interpretable by design, such as linear models and small trees. For complex models, **explainability tools** help.

- **Feature importance** shows which features the model relies on overall.
- **Partial dependence plots** show how predictions change as one feature varies.
- **SHAP values** break down an individual prediction into contributions from each feature: "income lowered this applicant's score by 0.12; late payments lowered it by 0.20."
- **LIME** fits a simple local model around one prediction to explain it.

Explanations build trust, help debug models, and are increasingly required by regulations.

Fairness and Bias

Machine learning models learn patterns from historical data, and history contains human bias. A model can therefore repeat or even amplify unfair treatment.

Where Bias Comes From

- **Historical bias:** if past hiring favoured one group, a model trained on past hiring decisions learns that preference.
- **Representation bias:** if a face dataset contains mostly light-skinned faces, the model performs worse on darker-skinned faces.
- **Measurement bias:** the data measures the wrong thing. Using "number of arrests" as a stand-in for "crime" reflects where police patrol, not only where crime happens.
- **Proxy variables:** even if you remove sensitive attributes such as gender or caste, other features, such as pin code or first name, can act as proxies for them.

Measuring Fairness

There are several definitions of fairness, and they cannot all be satisfied at once:

- **Demographic parity:** each group receives positive predictions at the same rate.
- **Equal opportunity:** qualified people in each group are approved at the same rate (equal true positive rates).

- **Equalised odds:** equal true positive *and* false positive rates across groups.
- **Calibration:** a predicted score of 0.8 means the same real-world chance for every group.

Choosing which definition matters is a human and social decision, not just a technical one.

Reducing Bias

- Collect more representative data.
- Audit model performance separately for different groups.
- Adjust training, for example by reweighting examples, or adjust thresholds per group where appropriate and lawful.
- Keep humans involved in high-stakes decisions.

Key idea: "The algorithm decided" is never an excuse. People choose the data, the objective and how predictions are used, and they are responsible for the outcomes.

Privacy

Models are often trained on personal data.

- Collect only the data you need (**data minimisation**) and get proper consent.

- **Anonymise** data, remembering that combinations of fields like age, pin code and gender can re-identify people.
- Models can **memorise** training data; large language models have been shown to repeat private text they were trained on.
- **Differential privacy** adds carefully calibrated noise so no individual's data can be inferred from the results.
- **Federated learning** trains models across many devices without collecting the raw data centrally, as with phone keyboards learning from typing.

Laws such as India's Digital Personal Data Protection Act and Europe's GDPR set legal requirements for handling personal data.

Safety, Security and Misuse

- **Adversarial examples:** tiny, carefully crafted changes to an input, invisible to humans, can fool a model, such as stickers that make a stop sign look like a speed-limit sign to a car's camera.
- **Data poisoning:** attackers insert bad examples into training data to corrupt a model.
- **Misuse:** the same technology can create deepfakes, automate scams or enable mass surveillance.

Responsible teams test models against such attacks, restrict access to powerful capabilities and think ahead about how a system could be misused.

Environmental Cost

Training very large models consumes a lot of electricity. Choosing efficient models, reusing pre-trained models through transfer learning, and not training bigger than necessary all reduce the environmental footprint.

A Responsible ML Checklist

Before deploying a model, ask:

1. Is machine learning actually needed, or would a simple rule do?
2. Is the training data representative of the people the model will affect?
3. How does performance differ across groups?
4. Can decisions be explained to the people affected?
5. What happens when the model is wrong, and can people appeal?
6. How will the model be monitored and updated?
7. Is personal data protected and used with consent?

Where to Go Next

- Practise on real datasets from Kaggle or the UCI repository.
- Learn a deep learning framework (PyTorch or TensorFlow) properly.
- Study the maths more deeply: linear algebra, probability and optimisation.
- Read about specialised fields that interest you: computer vision, natural language processing, recommender systems or reinforcement learning.
- Build and deploy a complete project end to end. It teaches more than any course.

Summary

- Production ML includes data pipelines, training pipelines, a model registry, serving and monitoring (MLOps).
- Predictions can be served in batch, online or on-device.
- Reproducibility needs versioned code, data, seeds and settings.
- Data drift and concept drift make models go stale; monitor and retrain.
- Explainability tools such as SHAP help justify predictions.
- Bias enters through data, measurement and proxies; fairness has several competing definitions.

- Protect privacy, defend against attacks and misuse, and consider environmental cost.

Practice Questions

1. What is the difference between data drift and concept drift?
Give an example of each.
2. When would you choose batch prediction over online prediction?
3. Explain how a model can be biased even if gender is removed from the features.
4. Why can't all definitions of fairness be satisfied at once?
Discuss briefly.
5. Apply the responsible ML checklist to a model that shortlists job applicants.